

DJANGO LAB GUIDE

Product Search QuerySet

Hands-on practice with QuerySets & Field Lookups

LAB GOAL

Build a QuerySet step by step to search products, refine the results, control ordering, and shape the returned data.

PROJECT

Django Project: product_search | App: products

Build the question first, then let the database answer it.

1. Lab Overview

WHAT WE BUILT

We started from `Product.objects`, created a `QuerySet` for active products, then refined it using stock, Q-based search, price limits, category, ordering, slicing, and finally `values()`.

FINAL SHAPE

The final result is not a full `Product` instance. It is a dictionary-like result containing `name`, `sku`, and `price`.

2. Lab Steps

1. Start with active Product records.

This step further refines the `QuerySet`.

```
products = Product.objects.filter(is_active=True)
```

2. Keep products whose stock is above zero.

This step further refines the `QuerySet`.

```
products = products.filter(stock__gt=0)
```

3. Search name OR SKU with Q.

This step further refines the `QuerySet`.

```
from django.db.models import Q

term = "audio"
products = products.filter(
    Q(name__icontains=term) | Q(sku__icontains=term)
)
```

4. Apply optional minimum and maximum prices.

This step further refines the QuerySet.

```
min_price = 50
max_price = 500

if min_price is not None:
    products = products.filter(price__gte=min_price)

if max_price is not None:
    products = products.filter(price__lte=max_price)
```

5. Filter by category name.

This step further refines the QuerySet.

```
products = products.filter(category__name="Audio")
```

6. Order by price, then name, then primary key.

This step further refines the QuerySet.

```
products = products.order_by("price", "name", "pk")
```

7. Return the first 10 results.

This step further refines the QuerySet.

```
products = products[:10]
```

8. Produce values() with name, SKU, and price.

This step further refines the QuerySet.

```
products = products.values("name", "sku", "price")
```

3. Core Concepts Used

QuerySet Mental Model

Manager → QuerySet → Refinement → Evaluation → Database → Results.

A QuerySet is built and refined first. The database is queried when evaluation is required.

Lookup Syntax

Use double underscores to express lookups and traverse relationships.

Examples: `price__gt`, `price__range`, `name__icontains`, `category__name`.

Chaining

Each filter/order/slice creates a refined QuerySet.

The original variable is not mutated by the refinement operation.

Q Objects

Use Q for OR, AND, NOT, and grouped conditions.

`Q(name__icontains=term) | Q(sku__icontains=term)`

F Expressions

Use F to compare stored fields or perform database-side arithmetic.

Example: `Product.objects.filter(stock__lt=F("reorder_level"))`

Result Control

Use `order_by()`, slicing, `distinct()`, `values()`, `values_list()`, `exists()`, and `count()`.

Choose the smallest result shape that the caller actually needs.

4. Full Query Flow

```
products = Product.objects.filter(is_active=True)
products = products.filter(stock__gt=0)

from django.db.models import Q
term = "audio"
products = products.filter(
    Q(name__icontains=term) | Q(sku__icontains=term)
)

min_price = 50
max_price = 500

if min_price is not None:
    products = products.filter(price__gte=min_price)

if max_price is not None:
    products = products.filter(price__lte=max_price)

products = products.filter(category__name="Audio")
products = products.order_by("price", "name", "pk")
products = products[:10]
products = products.values("name", "sku", "price")

list(products)
```

KEY IDEA

`filter()`, `order_by()`, `slicing`, and `values()` refine/build the `QuerySet`. `list(products)` requests the actual results and therefore evaluates it.

5. Evaluation & SQL Inspection

WHEN DOES A QUERYSET EXECUTE?

A `QuerySet` is lazy while you build it. Evaluation happens when results are requested, such as with `list()`, iteration, `count()`, `exists()`, `first()`, or `get()`.

HOW TO INSPECT THE SQL

If the `QuerySet` is stored in `products`, print the SQL Django generated with `products.query`.

```
print(products.query)
```

DEBUGGING NOTE

`result.query` caused a `NameError` because there was no variable named `result`. In this lab, the `QuerySet` variable is `products`, so the correct command is `print(products.query)`.

6. Exit Ticket — Did We Understand Evaluation?

TYPE	Examples used
Refine / Build	<code>filter()</code> — <code>order_by()</code> — <code>[:10]</code> — <code>values()</code>
Evaluate	<code>list(products)</code>

7. Final Checklist

✓ Created Django project: `product_search`

✓ Created app: `products`

✓ Created `Category` and `Product` models

✓ Ran migrations successfully

✓ Created sample categories and products

✓ Completed all 8 lab steps



Verified QuerySet evaluation



Verified SQL inspection command

8. Quick Review

What does `__gt` mean?

Answer: `>`

What does `icontains` change compared with `contains`?

Answer: Case-insensitive matching.

Why do we use `Q` here?

Answer: To express the OR between name and SKU.

What does `[:10]` do?

Answer: Limits the QuerySet to the first 10 results.

What does `values()` return?

Answer: Dictionary-like rows containing the selected fields.

Which line evaluates the QuerySet in our lab?

Answer: `list(products)`

How do you inspect generated SQL?

Answer: `print(products.query)`

LAB COMPLETE

Lab complete: 8/8 steps + Exit Ticket + SQL inspection.